PS — 5th Semester Practicals (All 15)

Practical 1: Study and Implementation of Advanced Data Structures in Python

Aim

To study and implement advanced data structures such as Stack, Queue, and Linked List in Python.

Theory

A Stack is a linear data structure that follows the LIFO (Last In First Out) principle, where insertion and deletion happen at one end called the top. The two primary operations are push (insert) and pop (remove), both performed in O(1) time; stacks are used in function call management, expression evaluation, undo mechanisms, and backtracking algorithms. A Queue follows the FIFO (First In First Out) principle, where insertion (enqueue) happens at the rear and deletion (dequeue) at the front; unlike a plain Python list, collections.deque is implemented as a doubly linked list internally, giving O(1) time complexity for operations at both ends instead of O(n) for a list. A Linked List is a linear collection of nodes where each node stores data and a reference (pointer) to the next node, allowing dynamic memory allocation and efficient insertion/deletion (O(1) once the position is known) unlike arrays, which require contiguous memory and O(n) shifting. Variants include singly linked lists (one-directional traversal, used here), doubly linked lists (traversal in both directions), and circular linked lists (last node points back to the first). Together, these three structures form the foundation for more advanced data structures such as trees, graphs, and hash tables.

Algorithm

1. Create an empty list to act as a stack.
2. Push elements onto the stack using append() and pop the last element using pop().
3. Create a deque to act as a queue.
4. Enqueue elements using append() and dequeue using popleft().
5. Create a linked list using nested dictionaries with 'data' and 'next' keys.
6. Traverse the linked list from head to tail, collecting values until next is None.
7. Display the results of all three data structures.

Program

```
stack = []
stack.append(10)
stack.append(20)
stack.append(30)
print("Stack:", stack)
print("Popped:", stack.pop())

from collections import deque
queue = deque()
queue.append("A")
queue.append("B")
queue.append("C")
print("Queue:", list(queue))
print("Dequeued:", queue.popleft())

node = {"data": 1, "next": {"data": 2, "next": {"data": 3, "next": None}}}
values = []
cur = node
while cur:
```

```
    values.append(cur["data"])
    cur = cur["next"]
print("Linked List:", values)
```

Output

```
Stack: [10, 20, 30]
Popped: 30
Queue: ['A', 'B', 'C']
Dequeued: A
Linked List: [1, 2, 3]
```

Result / Conclusion

The stack, queue, and linked list data structures were successfully implemented and their basic operations (push/pop, enqueue/dequeue, traversal) were verified.

Viva Questions

Q1. What is the difference between a Stack and a Queue?

Ans: A Stack follows LIFO (Last In First Out) — the last element inserted is the first to be removed (like a pile of plates). A Queue follows FIFO (First In First Out) — the first element inserted is the first to be removed (like a ticket line). Stack operations are push/pop at the top; Queue operations are enqueue at the rear and dequeue from the front.

Q2. Why is collections.deque preferred over a list for implementing a Queue in Python?

Ans: list.pop(0) (dequeue from the front) is O(n) because all remaining elements must be shifted one position left. collections.deque is backed by a doubly linked list and provides O(1) popleft() and append(), making it the correct choice for FIFO queues where both ends are accessed.

Q3. What is a node in a Linked List, and what does it contain?

Ans: A node is the basic unit of a linked list. It contains two parts: the data field (the actual value stored) and the next field (a reference/pointer to the next node in the sequence). The last node's next field is None, marking the end of the list.

Q4. What is the time complexity of push and pop operations on a Stack?

Ans: Both push (append to end) and pop (remove from end) on a Python list-based stack are O(1) amortised time, because Python lists use dynamic arrays with amortised constant-time appends and direct index access at the last position.

Q5. What are the real-world applications of a Stack?

Ans: Stacks are used in: (1) function call management — the call stack tracks active function frames; (2) expression evaluation and conversion (infix to postfix); (3) undo/redo mechanisms in text editors; (4) backtracking algorithms (maze solving, DFS); (5) browser history (back button).

Practical 2: Object-Oriented Programming Concepts

Aim

To study and implement Object-Oriented Programming concepts such as classes, objects, inheritance, and method overriding in Python.

Theory

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of objects, which bundle data (attributes) and behavior (methods) together, in contrast to procedural programming which separates data and functions. OOP rests on four main pillars: Encapsulation, which hides internal state and requires interaction through well-defined methods; Abstraction, which exposes only essential features while hiding implementation detail; Inheritance, which allows a class (child/subclass) to acquire properties and methods of another class (parent/superclass), promoting code reuse; and Polymorphism, which allows objects of different classes to be treated through a common interface. A class is a blueprint or template for creating objects, while an object is a concrete instance of a class with its own state. The __init__ method is a constructor, automatically invoked when an object is created, used to initialize instance attributes. Method overriding is a form of runtime polymorphism where a subclass provides a specific implementation of a method already defined in its parent class; when the method is called on a subclass object, Python's dynamic dispatch ensures the overridden version executes, which is demonstrated when Dog and Cat objects both respond differently to the same speak() call.

Algorithm

1. Define a base class Animal with an __init__ method and a speak() method.
2. Define subclasses Dog and Cat that inherit from Animal.
3. Override the speak() method in each subclass to return a specific sound.
4. Create a list of Animal objects containing Dog and Cat instances.
5. Loop through the list and call speak() on each object to demonstrate polymorphism.

Program

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return f"{self.name} makes a sound"

class Dog(Animal):
    def speak(self):
        return f"{self.name} barks"

class Cat(Animal):
    def speak(self):
        return f"{self.name} meows"

animals = [Dog("Rex"), Cat("Whiskers")]
for a in animals:
    print(a.speak())
```

Output

```
Rex barks
Whiskers meows
```

Result / Conclusion

OOP concepts including class definition, inheritance, and method overriding were successfully implemented and verified using Python.

Viva Questions

Q1. What are the four pillars of Object-Oriented Programming?

Ans: The four pillars are: (1) Encapsulation — bundling data and methods together and restricting direct access to internal state; (2) Abstraction — exposing only relevant features and hiding complexity; (3) Inheritance — a subclass acquires properties and methods from a parent class; (4) Polymorphism — the ability of different objects to respond to the same method call in different ways.

Q2. What is the purpose of the __init__ method in Python?

Ans: __init__ is the constructor method automatically called when a new object is created from a class. It initialises the instance's attributes (self.name = name, etc.). The self parameter refers to the newly created object itself, allowing attributes to be bound to that specific instance.

Q3. What is the difference between method overriding and method overloading?

Ans: Method overriding occurs when a subclass provides its own implementation of a method that is already defined in the parent class (same name, same signature) — it enables runtime polymorphism. Method overloading means defining multiple methods with the same name but different parameter counts/types — Python does not natively support overloading; the last definition wins; it is simulated using default arguments or *args.

Q4. What is the difference between a class and an object?

Ans: A class is a blueprint or template that defines attributes and methods (e.g., class Animal). An object (instance) is a concrete entity created from that class with its own state (e.g., Dog('Rex') creates one specific instance with name='Rex'). Multiple objects can be created from the same class, each with independent attribute values.

Q5. What is polymorphism and how is it demonstrated in this practical?

Ans: Polymorphism means 'many forms' — different objects respond to the same method call differently. In this practical, both Dog and Cat inherit from Animal and override speak(). When we loop over a list of Animal references and call a.speak(), Python dynamically dispatches to the correct subclass implementation: Dog prints 'barks' and Cat prints 'meows', even though both are called through the same interface.

Practical 3: File Handling and OS Module Operations

Aim

To study and implement file handling and OS module operations such as creating directories, reading/writing files, renaming, and deleting files in Python.

Theory

File handling in Python allows programs to create, read, write, append to, and manipulate files stored on persistent storage using the built-in open() function, which supports modes such as 'w' (write, truncates existing content), 'r' (read), 'a' (append), and 'x' (exclusive creation), along with binary variants like 'wb'/'rb'. Using a 'with' statement (context manager) ensures the file is automatically and safely closed even if an exception occurs, which is why it is preferred over manual open()/close() calls. The os module provides a cross-platform interface to interact with the underlying operating system: os.makedirs() creates a directory (and any missing intermediate directories, with exist_ok=True avoiding an error if it already exists), os.listdir() lists the names of entries in a directory, os.rename() renames or moves a file, os.remove() deletes a file, and os.rmdir() deletes an empty directory. Related modules such as os.path (for path manipulation and existence checks) and shutil (for recursive copy/move/delete of non-empty directories) are commonly used alongside the os module for complete file-system automation.

Algorithm

1. Import the os module.
2. Create a directory using os.makedirs().
3. Open a file in write mode and write text into it, then close it.
4. Open the same file in read mode and display its content.
5. List all files in the directory using os.listdir().
6. Rename the file using os.rename() and list the directory again.
7. Delete the file using os.remove() and delete the directory using os.rmdir().

Program

```
import os
os.makedirs("demo_folder", exist_ok=True)

with open("demo_folder/sample.txt", "w") as f:
    f.write("Hello, File Handling!\n")
    f.write("This is line two.")

with open("demo_folder/sample.txt", "r") as f:
    print("File Content:")
    print(f.read())

print("Files in folder:", os.listdir("demo_folder"))

os.rename("demo_folder/sample.txt", "demo_folder/renamed.txt")
print("After rename:", os.listdir("demo_folder"))

os.remove("demo_folder/renamed.txt")
os.rmdir("demo_folder")
print("Cleanup done")
```

Output

```
File Content:
Hello, File Handling!
This is line two.
Files in folder: ['sample.txt']
```

```
After rename: ['renamed.txt']
Cleanup done
```

Result / Conclusion

File handling and OS module operations (create, read, rename, delete) were successfully performed and verified.

Viva Questions

Q1. What are the different file opening modes in Python's open() function?

Ans: 'r' — read (default, file must exist); 'w' — write (creates or truncates); 'a' — append (creates or adds to end); 'x' — exclusive create (fails if file exists); 'b' — binary mode suffix (e.g. 'rb', 'wb'); '+' — read+write (e.g. 'r+', 'w+'). The mode determines whether the file pointer starts at the beginning or end and whether existing content is preserved.

Q2. Why should we use the 'with' statement when working with files?

Ans: The 'with' statement uses Python's context manager protocol (__enter__ and __exit__). It guarantees the file is closed automatically when the block exits, even if an exception is raised. Without it, forgetting to call f.close() can cause resource leaks (file handles not released), data corruption (buffered data not flushed), or errors when other processes try to access the file.

Q3. What is the difference between os.makedirs() and os.mkdir()?

Ans: os.mkdir() creates only a single directory and raises FileNotFoundError if any intermediate parent directory is missing. os.makedirs() creates the full path including all necessary intermediate directories (like mkdir -p in Unix). The exist_ok=True parameter suppresses the error if the directory already exists.

Q4. What is the difference between the os module and the shutil module?

Ans: The os module provides low-level, single-item file system operations: create/rename/delete individual files and empty directories. The shutil (shell utilities) module provides higher-level operations: shutil.copy() copies a file, shutil.move() moves/renames files across filesystems, and shutil.rmtree() deletes an entire directory tree (including non-empty directories), which os.rmdir() cannot do.

Q5. What happens if you open a file in 'w' mode and the file already exists?

Ans: Opening an existing file in 'w' (write) mode truncates it immediately — all existing content is erased and the file pointer is placed at position 0. If you want to add content without erasing, use 'a' (append) mode instead. If you want to create a file only if it does not already exist, use 'x' mode.

Practical 4: Exception Handling and Fetching Data from a REST API

Aim

To study and implement exception handling in Python while fetching data from a REST API.

Theory

Exception handling in Python is done using try, except, else, and finally blocks: code that might raise a runtime error is placed in the try block, the except block catches and handles specific (or general) exception types so the program does not crash abruptly, an optional else block runs only if no exception occurred, and the finally block always executes regardless of whether an exception was raised, making it ideal for cleanup actions like closing files or connections. Catching specific exception classes (e.g., URLError, TimeoutError) instead of a bare except is considered good practice as it avoids silently hiding unrelated bugs. A REST API (Representational State Transfer) is an architectural style that allows applications to communicate over HTTP using standard methods such as GET, POST, PUT, and DELETE, typically exchanging data in JSON (JavaScript Object Notation) format because it is lightweight and language-independent. The urllib.request module (a built-in alternative to third-party libraries like requests) can be used to send HTTP requests and read server responses, while the json module provides json.loads() to parse a JSON string/bytes into native Python objects (dicts/lists) and json.dumps() to convert Python objects back into a formatted JSON string.

Algorithm

1. Import the json and urllib.request modules.
2. Define the API URL to fetch data from.
3. Use a try block to open the URL and read the response with a timeout.
4. Parse the response using json.loads() and print it in readable format using json.dumps().
5. Use an except block to catch and print any errors during the request.
6. Use a finally block to indicate that the request has completed.

Program

```
import json
import urllib.request

try:
    url = "https://jsonplaceholder.typicode.com/todos/1"
    with urllib.request.urlopen(url, timeout=5) as response:
        data = json.loads(response.read())
        print("API Response:")
        print(json.dumps(data, indent=2))
except Exception as e:
    print("Error fetching data:", e)
finally:
    print("Request completed")
```

Output

```
API Response:
{
  "userId": 1,
  "id": 1,
  "title": "delectus aut autem",
  "completed": false
}
Request completed
```

Result / Conclusion

Exception handling was successfully used to safely fetch and parse JSON data from a REST API.

Viva Questions

Q1. What is the difference between try-except-else-finally blocks?

Ans: try: code that may raise an exception. except: runs only if an exception occurred in try — handles the error. else: runs only if NO exception occurred in try — for code that should execute on success. finally: always runs regardless of whether an exception occurred or not — used for cleanup (closing files, releasing locks). 'else' and 'finally' are optional.

Q2. What are the four main HTTP methods used in REST APIs?

Ans: GET — retrieve a resource (read-only, no side effects); POST — create a new resource; PUT/PATCH — update an existing resource (PUT replaces entirely, PATCH modifies partially); DELETE — remove a resource. These map directly to CRUD operations: Create=POST, Read=GET, Update=PUT/PATCH, Delete=DELETE.

Q3. What is the difference between JSON and XML?

Ans: JSON (JavaScript Object Notation) uses key-value pairs and arrays, is more compact, human-readable, and natively supported by JavaScript. XML (eXtensible Markup Language) uses opening/closing tags, supports attributes and namespaces, and is more verbose. JSON is preferred for REST APIs due to smaller payload size and easier parsing in most languages. XML is still used in SOAP web services and configuration files.

Q4. Why should you catch specific exceptions rather than using a bare 'except:'?

Ans: A bare 'except:' catches everything including KeyboardInterrupt, SystemExit, and MemoryError — errors you usually don't want to suppress. It can hide bugs by swallowing unexpected errors silently. Catching specific exceptions (e.g., urllib.error.URLError, json.JSONDecodeError) makes the code more maintainable, easier to debug, and ensures only expected error types are handled, letting genuinely unexpected errors propagate.

Q5. What is the difference between urllib.request and the requests library?

Ans: urllib.request is part of Python's standard library (no installation needed) but has a more verbose API. The requests library is a third-party package (pip install requests) with a much simpler, more Pythonic API: requests.get(url) vs urllib.request.urlopen(url). requests also automatically handles encoding, sessions, cookies, and authentication more easily. urllib is preferred when you want zero dependencies; requests is preferred for real-world applications.

Practical 5: Python Libraries and Basic Web Scraping

Aim

To study and implement basic web scraping using Python libraries requests and BeautifulSoup.

Theory

Web scraping is the process of programmatically extracting information from websites by fetching their raw HTML and parsing out the required data, commonly used for data collection, price monitoring, and research when no structured API is available; it should always be done in compliance with a site's terms of service and robots.txt. The requests library is a popular third-party HTTP client used to send GET/POST requests and retrieve a page's HTML content along with metadata like status codes and headers, and is generally preferred over the lower-level urllib for its simpler, more readable API. BeautifulSoup, from the bs4 package, is an HTML/XML parsing library that builds a navigable parse tree from raw markup and provides convenient methods to search and traverse it, such as tag access via dot notation (soup.title, soup.h1), find() and find_all() to locate elements by tag name, class, or attribute, and .text/.string to extract the readable text content of an element, abstracting away the need to manually write regular expressions against HTML.

Algorithm

1. Import the requests library and BeautifulSoup from bs4.
2. Send a GET request to the target URL to fetch the HTML page.
3. Parse the HTML content using BeautifulSoup with the 'html.parser'.
4. Extract and print the page title using soup.title.string.
5. Extract and print the first heading using soup.h1.text.
6. Find all paragraph tags using soup.find_all('p') and display their count and content.

Program

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com"
response = requests.get(url, timeout=5)
soup = BeautifulSoup(response.text, "html.parser")

print("Page Title:", soup.title.string)
print("First Heading:", soup.h1.text)
paragraphs = soup.find_all("p")
print("Number of paragraphs:", len(paragraphs))
print("First paragraph:", paragraphs[0].text)
```

Output

```
Page Title: Example Domain
First Heading: Example Domain
Number of paragraphs: 2
First paragraph: This domain is for use in documentation examples without needing permission. Avoid use in..
```

Result / Conclusion

Basic web scraping was successfully implemented, extracting the title, heading, and paragraph content from a web page.

Viva Questions

Q1. What is web scraping and what are its legal/ethical considerations?

Ans: Web scraping is the automated extraction of data from websites by fetching HTML and parsing it programmatically. Ethical/legal considerations: always check the site's robots.txt file (which lists paths scrapers should not access) and Terms of Service; avoid overloading servers with rapid requests; do not scrape personal or copyrighted data without permission; use official APIs when available. Violating these can lead to IP bans, legal action, or violation of laws like the CFAA.

Q2. What is the difference between soup.find() and soup.find_all()?

Ans: soup.find(tag) returns the first matching element as a Tag object (or None if not found). soup.find_all(tag) returns a list of ALL matching elements (empty list if none found). find() is equivalent to find_all(..., limit=1)[0]. Both accept tag name, CSS class, id, or any attribute as search criteria. find() is used when you expect a unique element; find_all() when you need all occurrences.

Q3. What parsers can BeautifulSoup use, and which is recommended?

Ans: 'html.parser' is Python's built-in HTML parser — no extra install needed, reasonably fast, lenient with malformed HTML. 'lxml' is a fast C-based parser (pip install lxml) — best performance for large pages. 'html5lib' (pip install html5lib) parses exactly like a browser — slowest but most accurate for malformed HTML. For most tasks, 'html.parser' is sufficient; use 'lxml' for performance-critical scraping.

Q4. What is the difference between .text and .string in BeautifulSoup?

Ans: .text (or .get_text()) returns all the text content of a tag and all its descendants concatenated as a single string, stripping HTML tags. .string returns the NavigableString of a tag ONLY if it contains a single string with no nested tags; it returns None if the tag has multiple children. Use .text for general extraction; .string only when you are certain the tag has a single text node.

Q5. How can you handle a failed HTTP request in web scraping?

Ans: Check response.status_code: 200 = OK, 404 = Not Found, 403 = Forbidden, 429 = Too Many Requests. Use response.raise_for_status() to raise an HTTPError for 4xx/5xx responses. Wrap requests.get() in a try-except block for connection errors (requests.exceptions.ConnectionError, Timeout). Add retry logic with exponential backoff for rate-limited sites. Always set a timeout parameter to avoid hanging indefinitely.

Practical 6: Arrays, Strings, Linked Lists, Stacks and Queues

Aim

To study and implement basic operations on arrays, strings, linked lists, stacks, and queues in Python.

Theory

Arrays (implemented as lists in Python) store elements in contiguous, indexable memory locations, giving O(1) random access by index; appending to the end is amortized O(1), while inserting/deleting at arbitrary positions is O(n) due to element shifting. Strings in Python are immutable sequences of characters, meaning any operation that appears to modify a string (such as reversal) actually creates and returns a new string object; slicing with a step of -1 (s[::-1]) is a concise idiom for reversal because it walks the sequence from the last index to the first. A Stack enforces LIFO (Last In First Out) order, supporting push (insert at top) and pop (remove from top) in O(1) time, useful for reversing sequences, expression parsing, and recursion simulation. A Queue enforces FIFO (First In First Out) order, supporting enqueue (insert at rear) and dequeue (remove from front); collections.deque is preferred over a plain list for queues because it is backed by a doubly linked structure, giving O(1) complexity for operations at both ends versus O(n) for list.pop(0).

Algorithm

1. Create a list (array) and append a new element to it.
2. Reverse a given string using slicing (s[::-1]).
3. Create a stack using a list, extend it with elements, and pop the last element.
4. Create a queue using deque, initialize it with elements, and dequeue from the front.
5. Print the results after each operation.

Program

```
arr = [5, 2, 8, 1]
arr.append(9)
print("Array:", arr)

s = "Hello World"
print("Reversed String:", s[::-1])

stack = []
stack.extend([1, 2, 3])
stack.pop()
print("Stack after pop:", stack)

from collections import deque
queue = deque([1, 2, 3])
queue.popleft()
print("Queue after dequeue:", list(queue))
```

Output

```
Array: [5, 2, 8, 1, 9]
Reversed String: dlroW olleH
Stack after pop: [1, 2]
Queue after dequeue: [2, 3]
```

Result / Conclusion

Basic operations on arrays, strings, stacks, and queues were successfully implemented and verified.

Viva Questions

Q1. What is the difference between a Python list and a traditional array?

Ans: A traditional array (e.g., in C) stores elements of the same data type in contiguous memory, providing fixed size and direct memory access. Python lists are dynamic arrays that can store heterogeneous types (int, str, objects), automatically resize, and support O(1) append (amortised). Python's array module or NumPy arrays provide typed arrays closer to traditional ones. For numeric computation, NumPy arrays are more memory-efficient and faster.

Q2. Why are Python strings immutable, and what are the implications?

Ans: Strings are immutable because Python interns (caches) string objects for memory efficiency and thread safety — multiple variables can safely reference the same string object. Implications: any 'modification' (concatenation, slicing, upper()) creates a new string object. For performance when building strings in a loop, use a list and ''.join(parts) instead of repeated += which creates O(n^2) temporary objects.

Q3. What does the slicing notation s[::-1] do?

Ans: s[start:stop:step] is Python's slice syntax. s[::-1] uses start=None (end of string), stop=None (beginning of string), and step=-1 (move backwards one character at a time). This traverses the string from the last character to the first, effectively reversing it. It returns a new string object since strings are immutable.

Q4. What is the time complexity of inserting at the beginning of a Python list vs a deque?

Ans: Inserting at the beginning of a Python list with list.insert(0, x) is O(n) because all existing elements must be shifted one position to the right to make room. deque.appendleft(x) is O(1) because a deque is implemented as a doubly-linked list, allowing constant-time insertion and deletion at both ends without shifting.

Q5. How does Python's list dynamically resize?

Ans: Python lists use a dynamic array (over-allocation) strategy. When the internal buffer is full, Python allocates a new, larger buffer (roughly 1.125x growth factor) and copies all existing elements into it. This makes individual appends O(n) in the worst case but O(1) amortised over many appends, since resizing happens infrequently. The list object tracks both the logical length (len()) and the allocated capacity separately.

Practical 7: Tree and Graph Data Structures with DFS and BFS

Aim

To study and implement graph traversal techniques - Breadth-First Search (BFS) and Depth-First Search (DFS).

Theory

A Tree is a hierarchical, acyclic connected graph with a single root node, where every node (except the root) has exactly one parent, commonly used to represent hierarchical relationships such as file systems and organization charts. A Graph is a more general data structure consisting of nodes (vertices) connected by edges, which may be directed or undirected, weighted or unweighted, and can contain cycles; it is often represented using an adjacency list (a dictionary mapping each node to its neighbors, as used here) or an adjacency matrix. Breadth-First Search (BFS) explores a graph level by level using a queue (FIFO), visiting all neighbors of the current node before moving to the next level, which makes it well suited for finding the shortest path in an unweighted graph. Depth-First Search (DFS) explores as far as possible along each branch before backtracking, implemented either recursively (using the call stack) or iteratively with an explicit stack, and is useful for tasks like cycle detection and topological sorting. Both algorithms maintain a 'visited' set to avoid revisiting nodes and getting stuck in infinite loops when cycles are present, and both run in O(V + E) time, where V is the number of vertices and E is the number of edges.

Algorithm

1. Represent the graph as a dictionary of adjacency lists.

2. For BFS: initialize a visited set and a queue with the start node; repeatedly dequeue a node, record it, and enqueue its unvisited neighbors.

3. For DFS: recursively visit the start node, mark it visited, record it, and recurse into its unvisited neighbors.

4. Call bfs() and dfs() from the same starting node and print the traversal order for each.

Program

```
graph = {
    'A': ['B', 'C'],
    'B': ['D'],
    'C': ['D'],
    'D': []
}

def bfs(start):
    visited, queue, order = {start}, [start], []
    while queue:
        node = queue.pop(0)
        order.append(node)
        for n in graph[node]:
            if n not in visited:
                visited.add(n)
                queue.append(n)
    return order

def dfs(node, visited=None, order=None):
    if visited is None:
        visited, order = set(), []
    visited.add(node)
    order.append(node)
    for n in graph[node]:
        if n not in visited:
            dfs(n, visited, order)
    return order
print("BFS Traversal:", bfs('A'))
```

```
print("DFS Traversal:", dfs('A'))
```

Output

```
BFS Traversal: ['A', 'B', 'C', 'D']
DFS Traversal: ['A', 'B', 'D', 'C']
```

Result / Conclusion

BFS and DFS graph traversal algorithms were successfully implemented and their traversal orders were verified.

Viva Questions

Q1. What is the key difference between BFS and DFS?

Ans: BFS (Breadth-First Search) uses a queue (FIFO) and explores all neighbors of a node before going deeper — it visits level by level. DFS (Depth-First Search) uses a stack (or recursion) and explores as far down one branch as possible before backtracking. BFS finds the shortest path in an unweighted graph; DFS is better for tasks like cycle detection, topological sorting, and solving mazes.

Q2. What is an adjacency list and how does it compare to an adjacency matrix?

Ans: An adjacency list stores each vertex's neighbors as a list/set (e.g., a dictionary). Space: O(V + E). Efficient for sparse graphs. An adjacency matrix uses a V×V grid where matrix[i][j]=1 if edge (i,j) exists. Space: O(V^2). Efficient for dense graphs and O(1) edge lookup. For most real-world graphs (sparse), adjacency lists are preferred.

Q3. Why is a 'visited' set necessary in graph traversal?

Ans: Unlike trees, graphs can contain cycles (e.g., A→B→C→A). Without tracking visited nodes, the traversal would revisit nodes indefinitely, causing an infinite loop. The visited set records nodes already processed so they are skipped on re-encounter. For trees (which are acyclic), a visited set is not strictly necessary but is good practice.

Q4. What is the time complexity of BFS and DFS?

Ans: Both BFS and DFS have time complexity O(V + E), where V is the number of vertices and E is the number of edges. Each vertex is visited once (O(V)) and each edge is examined once (O(E)). Space complexity: BFS uses O(V) for the queue (all nodes at the widest level); DFS uses O(V) for the recursion call stack (depth of the graph in the worst case).

Q5. Give a real-world application of BFS and DFS each.

Ans: BFS applications: shortest path in unweighted graphs (GPS navigation, social network friend suggestions at distance 1, 2, 3), level-order traversal of a tree, web crawlers. DFS applications: detecting cycles in a graph, topological sorting (dependency resolution), solving puzzles like mazes and Sudoku, finding connected components, generating permutations.

Practical 8: Sorting and Searching Algorithms

Aim

To study and implement Bubble Sort and Binary Search algorithms in Python.

Theory

Sorting algorithms arrange elements of a list into a defined order (typically ascending or descending) and are foundational to efficient searching and data processing. Bubble Sort is a simple comparison-based sorting algorithm that repeatedly steps through the list, compares each pair of adjacent elements, and swaps them if they are in the wrong order; after each full pass, the largest unsorted element 'bubbles up' to its correct position, giving a time complexity of O(n^2) in the average and worst case and O(n) in the best case (already sorted) with an early-exit optimization, and O(1) space complexity since sorting happens in place. Searching algorithms locate a target value within a collection. Linear Search checks elements sequentially (O(n)), while Binary Search is a much more efficient divide-and-conquer algorithm for finding an item in a sorted array: it repeatedly compares the target with the middle element and discards the half of the array that cannot contain the target, narrowing the search range until the element is found or the range becomes empty, achieving a time complexity of O(log n). Binary Search requires the input array to be sorted beforehand, which is why it is applied here after bubble_sort() has arranged the data.

Algorithm

1. Bubble Sort: for each pass, compare adjacent elements and swap them if the left is greater than the right; repeat until no swaps are needed.

2. Binary Search: set low and high pointers; compute the mid index; compare the middle element with the target; narrow the search range accordingly until found or the range is empty.

3. Sort the given array using bubble_sort().

4. Search for a target value in the sorted array using binary_search() and print its index.

Program

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

data = [64, 25, 12, 22, 11]
sorted_data = bubble_sort(data.copy())
print("Sorted Array:", sorted_data)
print("Index of 22:", binary_search(sorted_data, 22))
```

Output

```
Sorted Array: [11, 12, 22, 25, 64]
```

```
Index of 22: 2
```

Result / Conclusion

Bubble Sort and Binary Search were successfully implemented, sorting the array and locating the target element correctly.

Viva Questions

Q1. What is the time complexity of Bubble Sort in best, average, and worst cases?

Ans: Worst case (reverse sorted): $O(n^2)$ — every element must be swapped in every pass. Average case: $O(n^2)$. Best case (already sorted, with early-exit optimisation): O(n) — one pass with zero swaps detects sorted order. Space complexity: O(1) — in-place sorting with only a constant number of extra variables.

Q2. What is the prerequisite for Binary Search?

Ans: The array must be sorted in ascending (or descending, with the comparison adjusted) order before applying Binary Search. If the array is unsorted, Binary Search may miss the target entirely and return a wrong result. The cost of sorting (e.g., O(n log n) with merge sort) is worthwhile only if many searches will be performed on the same dataset.

Q3. How does Binary Search achieve O(log n) time complexity?

Ans: Binary Search halves the search space at each step. Starting with n elements: after 1 comparison n/2 remain, after 2 comparisons n/4 remain, and so on. After k comparisons, n/2^k elements remain. The search ends when n/2^k = 1, i.e., k = log■(n). So the maximum number of comparisons is log■(n), giving O(log n) time complexity.

Q4. Is Bubble Sort a stable sorting algorithm? What does 'stable' mean?

Ans: Yes, Bubble Sort is stable. A stable sorting algorithm preserves the relative order of equal elements — if two elements have the same key, the one that appeared first in the original array appears first in the sorted output. Bubble Sort is stable because it only swaps adjacent elements when strictly greater-than (not equal), so equal elements never swap past each other.

Q5. What are some more efficient alternatives to Bubble Sort?

Ans: Merge Sort: O(n log n) all cases, stable, uses O(n) extra space. Quick Sort: O(n log n) average, $O(n^2)$ worst case, in-place (O(log n) stack). Heap Sort: O(n log n) all cases, in-place, not stable. Python's built-in sorted() uses Timsort — O(n log n) worst, O(n) best, stable, hybrid of merge sort and insertion sort. Bubble Sort is mainly used for educational purposes, not in production.

Practical 9: Time and Space Complexity Analysis

Aim

To study and analyze the time and space complexity of Linear Search and Binary Search algorithms.

Theory

Time complexity measures how the running time of an algorithm grows as a function of input size (n), typically expressed using Big-O notation, which describes the worst-case upper bound on growth while ignoring constant factors and lower-order terms; common classes in increasing order of growth include O(1), O(log n), O(n), O(n log n), and O(n^2). Space complexity measures the additional (auxiliary) memory an algorithm requires beyond the input itself, which is important when working with large datasets or memory-constrained systems. Linear Search checks each element of the array sequentially until it finds a match or reaches the end, giving a worst-case time complexity of O(n) since it may need to examine every element. Binary Search works only on sorted data and repeatedly halves the search range by comparing the target with the middle element, giving a much better time complexity of O(log n), which scales far more efficiently as the dataset grows large - this is empirically demonstrated in this practical by timing both algorithms on a dataset of 100,000 elements. Both algorithms use O(1) additional space since they operate in-place using only a fixed number of variables regardless of input size, rather than allocating new data structures proportional to n.

Algorithm

1. Define linear_search() which iterates through the array checking each element against the target.
2. Define binary_search() which divides the sorted array range in half repeatedly.
3. Generate a large list of data and choose a target value.
4. Measure execution time of linear_search() using time.perf_counter().
5. Measure execution time of binary_search() similarly.
6. Print both execution times along with their theoretical time and space complexities.

Program

```
import time

def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

data = list(range(100000))
target = 99999

start = time.perf_counter()
linear_search(data, target)
t1 = time.perf_counter() - start
start = time.perf_counter()
```

```
binary_search(data, target)
t2 = time.perf_counter() - start

print(f"Linear Search Time: {t1:.6f} s  (Complexity: O(n))")
print(f"Binary Search Time: {t2:.6f} s  (Complexity: O(log n))")
print("Space Complexity: O(1) for both (in-place)")
```

Output

```
Linear Search Time: 0.002474 s  (Complexity: O(n))
Binary Search Time: 0.000005 s  (Complexity: O(log n))
Space Complexity: O(1) for both (in-place)
```

Result / Conclusion

The time and space complexity of Linear Search and Binary Search were analyzed and empirically verified, confirming Binary Search is significantly faster on large sorted datasets.

Viva Questions

Q1. What is Big-O notation and what does it represent?

Ans: Big-O notation describes the upper bound (worst-case) growth rate of an algorithm's time or space requirements as the input size n increases, ignoring constant factors and lower-order terms. For example, O(n) means the time grows linearly with n; O(n²) means it grows quadratically. It helps compare algorithms independently of hardware or implementation details. Other notations: Ω (best case), Θ (tight/average case).

Q2. Why does Binary Search have O(log n) time complexity?

Ans: Each comparison in Binary Search eliminates half the remaining elements. Starting with n elements, after 1 step n/2 remain, after 2 steps n/4, after k steps n/2^k. The algorithm terminates when 1 element remains: n/2^k = 1 ■ k = log■(n). So the maximum number of comparisons is proportional to log■(n), giving O(log n) time complexity.

Q3. What is the difference between time complexity and space complexity?

Ans: Time complexity measures how the number of operations (running time) grows with input size n. Space complexity measures how much additional memory (auxiliary space) an algorithm uses beyond the input itself, also as a function of n. An algorithm can be fast but memory-hungry (e.g., merge sort: O(n log n) time, O(n) space) or slow but memory-efficient (e.g., bubble sort: O(n²) time, O(1) space).

Q4. What does O(1) space complexity mean for both search algorithms?

Ans: O(1) space complexity means the algorithm uses a constant amount of additional memory regardless of the input size n. Both Linear Search and Binary Search only use a fixed number of scalar variables (loop counter i, or low/high/mid pointers) — no additional arrays, lists, or data structures are allocated proportional to n. The input array itself is not counted as auxiliary space.

Q5. When would you prefer Linear Search over Binary Search?

Ans: Linear Search is preferred when: (1) the data is unsorted — sorting first adds O(n log n) cost that outweighs the benefit for a single search; (2) the dataset is very small (n < ~20) — the constant overhead of Binary Search matters; (3) searching through a linked list — you cannot do random O(1) access needed for Binary Search; (4) searching on a condition other than equality (e.g., first element satisfying a predicate).

Practical 10: Recursion and Backtracking

Aim

To study and implement recursion using a Factorial program and backtracking using the N-Queens problem in Python.

Theory

Recursion is a programming technique where a function calls itself, directly or indirectly, to solve smaller instances of the same problem, until it reaches a base case that can be solved directly without further recursion; the factorial function demonstrates this, with factorial(0) = 1 as the base case and factorial(n) = n * factorial(n-1) as the recursive case, with each call pushing a new frame onto the call stack until the base case unwinds the recursion. Backtracking is an algorithmic technique for solving constraint-satisfaction problems incrementally, by building a solution one step at a time and abandoning ('backtracking' from) a partial solution as soon as it is determined that it cannot possibly lead to a valid complete solution, which prunes the search space far more efficiently than checking every possible combination (brute force). The N-Queens problem is a classic backtracking example: it requires placing N queens on an N x N chessboard such that no two queens attack each other (i.e., no two share the same row, column, or diagonal); the algorithm places a queen in a row, checks safety against all previously placed queens using is_safe(), recurses to the next row if safe, and backtracks (removes the queen and tries the next column) if no column in the current row is safe.

Algorithm

1. Factorial: if n is 0 return 1, otherwise return n multiplied by factorial(n-1).

2. N-Queens: maintain a board array storing the column position of the queen in each row.

3. Define is_safe() to check if placing a queen at a given row/column conflicts with previously placed queens.

4. Define backtrack() which tries placing a queen in each column of the current row; if safe, recurse to the next row, otherwise backtrack.

5. When all rows are filled, save the solution.

6. Print the factorial result and the number/first solution of the N-Queens problem.

Program

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)

def solve_nqueens(n):
    solutions = []
    board = [-1] * n

    def is_safe(row, col):
        for r in range(row):
            if board[r] == col or abs(board[r] - col) == abs(r - row):
                return False
        return True

    def backtrack(row):
        if row == n:
            solutions.append(board.copy())
            return
        for col in range(n):
            if is_safe(row, col):
                board[row] = col
                backtrack(row + 1)
                board[row] = -1
    backtrack(0)
```

```
    return solutions

print("Factorial of 5:", factorial(5))

solutions = solve_nqueens(4)
print(f"Number of solutions for 4-Queens: {len(solutions)}")
print("First solution (col positions per row):", solutions[0])
```

Output

```
Factorial of 5: 120
Number of solutions for 4-Queens: 2
First solution (col positions per row): [1, 3, 0, 2]
```

Result / Conclusion

Recursion and backtracking were successfully implemented and demonstrated using the Factorial and N-Queens programs.

Viva Questions

Q1. What are the two essential components of every recursive function?

Ans: 1. Base case: a condition under which the function returns a result directly without calling itself (e.g., factorial(0) = 1). It stops the recursion and prevents infinite loops. 2. Recursive case: the function calls itself with a simpler/smaller version of the problem (e.g., n * factorial(n-1)), making progress toward the base case.

Q2. What is a stack overflow in the context of recursion?

Ans: Each recursive call adds a new frame (with local variables and the return address) to the call stack. If recursion is too deep (no base case, or n is very large), the call stack exhausts available memory, causing a stack overflow (RecursionError in Python: 'maximum recursion depth exceeded'). Python's default recursion limit is 1000. sys.setrecursionlimit() can increase it, but deep recursion should be converted to iteration for safety.

Q3. What is backtracking and how does it differ from brute force?

Ans: Brute force generates all possible solutions and checks each one — extremely slow for large search spaces. Backtracking builds the solution incrementally and abandons (backtracks from) a partial solution as soon as a constraint violation is detected, without exploring the remaining subtree. This pruning makes backtracking much faster than brute force for constraint-satisfaction problems like N-Queens, Sudoku, and crossword puzzles.

Q4. How does the is_safe() function check for diagonal conflicts in N-Queens?

Ans: Two queens at (r1, c1) and (r2, c2) are on the same diagonal if and only if |r1 - r2| == |c1 - c2| (the row distance equals the column distance). is_safe(row, col) checks for three conflicts with each previously placed queen at board[r]: same column (board[r] == col), or same diagonal (abs(board[r] - col) == abs(r - row)). Row conflicts are impossible since each row places exactly one queen.

Q5. What is the difference between direct and indirect recursion?

Ans: Direct recursion: a function calls itself directly (e.g., factorial calls factorial). Indirect recursion: function A calls function B which calls function A (e.g., isEven(n) calls isOdd(n-1) which calls isEven(n-2)). Both require proper base cases to terminate. Mutual recursion (indirect) is sometimes used for alternating state machines but is generally harder to reason about and optimise.

Practical 11: Responsive Web Pages using HTML, CSS and JavaScript

Aim

To design and implement a responsive web page using HTML, CSS, and JavaScript.

Theory

HTML (HyperText Markup Language) provides the semantic structure and content of a web page using nested elements (tags) such as div, h2, p, and button. CSS (Cascading Style Sheets) controls the presentation of that structure, including layout, colors, spacing, fonts, and visual effects like box-shadow and border-radius, and can be applied inline, via an internal <style> block, or via an external stylesheet. JavaScript adds interactivity and dynamic behavior to an otherwise static page by manipulating the Document Object Model (DOM) - the browser's in-memory tree representation of the HTML - at runtime, for example by changing an element's innerText or innerHTML in response to user actions. Responsive Web Design ensures a page's layout automatically adjusts to fit different screen sizes and devices (desktop, tablet, mobile) without requiring separate versions of the site, commonly achieved using relative units (%, em, rem) instead of fixed pixels, flexible containers (Flexbox/Grid), media queries, and constraints like max-width so content neither overflows small screens nor stretches awkwardly on large ones. JavaScript event handlers, such as the onclick attribute or addEventListener(), allow the page to respond dynamically to user actions like clicks, input, or form submission without requiring a full page reload.

Algorithm

1. Create the HTML structure with a container div styled as a 'card'.

2. Use CSS to style the body and card with background color, padding, border-radius, and box-shadow, and set a max-width on the card for responsiveness.

3. Add a button element with an onclick JavaScript handler.

4. On button click, use JavaScript to update the innerText of a paragraph element to show a message.

5. Open the page in a browser and verify the layout adjusts to screen width and the button click updates the message.

Program

```
<!DOCTYPE html>
<html>
<head>
<style>
  body { font-family: Arial; background: #f0f2f5; margin: 0; }
  .card { max-width: 400px; margin: 40px auto; padding: 20px;
          background: white; border-radius: 10px; box-shadow: 0 2px 8px #ccc; text-align: center; }
  button { padding: 8px 16px; background: #4CAF50; color: white; border: none; border-radius: 5px; }
</style>
</head>
<body>
  <div class="card">
    <h2>Responsive Card</h2>
    <p>This page resizes with the screen.</p>
    <button onclick="document.getElementById('msg').innerText='Button Clicked!'">Click Me</button>
    <p id="msg"></p>
  </div>
</body>
</html>
```

Output

```
Browser renders a centered white card on a light grey
```

```
background with heading "Responsive Card", a paragraph,
and a green "Click Me" button.

On clicking the button, the page updates and shows:
  Button Clicked!

The card layout adjusts automatically to fit any
screen width (responsive design).
```

Result / Conclusion

A responsive web page was successfully designed and implemented using HTML, CSS, and JavaScript, with verified interactivity and layout adjustment.

Viva Questions

Q1. What is the difference between inline, internal, and external CSS?

Ans: Inline CSS: style attribute directly on an HTML element (<p style='color:red'>). Highest specificity, hardest to maintain. Internal CSS: <style> block inside the <head> — applies to one page, good for small prototypes. External CSS: a separate .css file linked via <link rel='stylesheet'>. Best practice for real projects — separates concerns, enables caching, shared across multiple pages. Specificity order: inline > ID > class > tag.

Q2. What is the Document Object Model (DOM)?

Ans: The DOM is a tree-structured, in-memory representation of an HTML document created by the browser when a page loads. Each HTML element becomes a node in the tree. JavaScript can access and manipulate this tree via the document object (e.g., document.getElementById(), element.innerText = '...'). Changes to the DOM are immediately reflected in the browser without reloading the page, enabling dynamic interactivity.

Q3. What is responsive web design and what techniques achieve it?

Ans: Responsive web design (RWD) makes a webpage layout adapt to different screen sizes (desktop, tablet, mobile) automatically. Key techniques: (1) fluid grids using % widths instead of fixed px; (2) flexible images (max-width: 100%); (3) CSS media queries (@media (max-width: 768px) {...}) to apply different styles at breakpoints; (4) CSS Flexbox and Grid for adaptive layouts; (5) the viewport meta tag (<meta name='viewport' content='width=device-width, initial-scale=1'>).

Q4. What is the difference between innerText and innerHTML?

Ans: innerText sets or gets the visible text content of an element, treating the value as plain text — any HTML tags in the string are escaped and displayed literally. innerHTML sets or gets the HTML markup inside an element — HTML tags are parsed and rendered by the browser. Use innerText for plain text to avoid XSS vulnerabilities; use innerHTML only when you deliberately want to insert HTML markup and have sanitised the input.

Q5. What is the CSS Box Model?

Ans: The CSS Box Model describes how each HTML element is rendered as a rectangular box consisting of four layers from inside out: Content (the actual text/image), Padding (space between content and border, transparent), Border (visible line around the padding), Margin (transparent space outside the border, separating elements). The default box-sizing is content-box (width/height apply to content only); box-sizing: border-box includes padding and border in the width/height calculation, which is more intuitive for layout.

Practical 12: React Basics and Component Architecture

Aim

To study and implement basic React concepts such as components, props, and state using a Counter application.

Theory

React is a declarative, component-based JavaScript library (developed by Meta) for building user interfaces out of small, reusable, self-contained pieces called components, instead of manipulating the DOM directly and imperatively as in vanilla JavaScript. A functional component is simply a JavaScript function that accepts props (inputs) and returns JSX - a syntax extension that lets HTML-like markup be written directly inside JavaScript, which is compiled (transpiled) into regular JavaScript function calls (React.createElement) by tools like Babel. Props (short for properties) are read-only data passed from a parent component to a child component, enabling component reuse with different inputs. The useState hook is one of React's built-in Hooks, which let functional components hold local, mutable state without needing class components; calling its setter function (e.g., setCount) does not mutate the variable in place but instead schedules a re-render of the component with the updated value, and React efficiently updates only the changed parts of the real DOM using its Virtual DOM diffing algorithm. Component composition, as seen with the Counter component nested inside the App component, allows complex UIs to be built by combining small, independently testable pieces, one of React's core architectural strengths.

Algorithm

1. Import React and the useState hook.
2. Define a Counter component that initializes a count state variable to 0 using useState.
3. Render the current count and a button that calls setCount to increment the count on click.
4. Define an App component that renders a heading and includes the Counter component.
5. Export App as the default component and render it in the browser.
6. Verify that each click on the Increment button updates the displayed count.

Program

```
import React, { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}

function App() {
  return (
    <div>
      <h1>React Basics Demo</h1>
      <Counter />
    </div>
  );
}

export default App;
```

Output

```
Browser renders:
  React Basics Demo
  Count: 0
  [ Increment ]

Each click on "Increment" re-renders the Counter
component and increases the displayed count by 1:
  Count: 1
  Count: 2
  Count: 3 ...
```

Result / Conclusion

Basic React concepts - components, props, and state management using useState - were successfully implemented and verified in the Counter application.

Viva Questions

Q1. What is the Virtual DOM and why does React use it?

Ans: The Virtual DOM is a lightweight JavaScript copy of the real DOM. When state changes, React creates a new Virtual DOM tree and diffs it against the previous one (reconciliation), identifying the minimal set of real DOM changes needed. It then applies only those changes (patching), which is much faster than re-rendering the entire page. This makes React updates efficient, even for complex UIs with many components.

Q2. What is the difference between props and state in React?

Ans: Props (properties) are read-only inputs passed from a parent component to a child — the child cannot modify them. They are used for component configuration and data flow downwards. State is mutable data managed inside a component using useState — when state changes, React re-renders that component. Props flow down; state is local. State changes trigger re-renders; prop changes cause the parent to re-render, which may update the child's props.

Q3. What is JSX and how does it work?

Ans: JSX (JavaScript XML) is a syntax extension that lets you write HTML-like markup inside JavaScript code. It is NOT valid JavaScript — a transpiler (Babel) converts JSX to React.createElement() calls. For example, <h1>Hello</h1> becomes React.createElement('h1', null, 'Hello'). JSX allows embedding JavaScript expressions using curly braces {expression}. It makes component code more readable and maintainable than raw React.createElement() calls.

Q4. What does useState return and how do you use it?

Ans: useState(initialValue) returns an array of exactly two elements: [currentValue, setterFunction]. Destructuring: const [count, setCount] = useState(0). count holds the current state value (0 initially). setCount(newValue) schedules a re-render with the new value — it does NOT mutate count directly. Each component instance has its own independent state, and state persists across re-renders (unlike regular variables which reset).

Q5. What is the difference between React and vanilla JavaScript for building UIs?

Ans: Vanilla JS manually manipulates the DOM (document.getElementById, innerHTML), which becomes complex and error-prone for large, dynamic UIs. React uses declarative programming — you describe what the UI should look like for a given state, and React handles DOM updates. React provides: component reuse, unidirectional data flow, efficient updates via Virtual DOM, and a rich ecosystem. Trade-off: React adds a framework overhead (~40KB); vanilla JS is lighter for simple pages.

Practical 13: Web Application using Flask Framework

Aim

To study and implement a simple web application using the Flask framework in Python.

Theory

Flask is a lightweight, minimalistic Python 'micro' web framework built on top of the Werkzeug WSGI toolkit and Jinja2 templating engine, used to build web applications and RESTful APIs with minimal boilerplate compared to full-stack frameworks like Django. A Flask application is represented by an instance of the Flask class, typically created with app = Flask(__name__), where __name__ helps Flask locate resources relative to the module. Route decorators (@app.route(path)) bind a URL path (and optionally HTTP methods like GET/POST) to a Python function known as a view function; when a client requests that path, Flask invokes the corresponding view function and sends whatever it returns (a string, JSON, or rendered HTML template) back as the HTTP response body. Multiple routes allow a single application to serve different pages or API endpoints, as demonstrated by the '/' and '/about' routes. The app.run() method starts Flask's built-in development server (not intended for production use) to listen for and serve incoming HTTP requests on a local host and port, with debug=True enabling auto-reload on code changes and detailed in-browser error tracebacks during development.

Algorithm

1. Import Flask and create a Flask application instance.
2. Define a route '/' mapped to a home() function that returns a welcome message.
3. Define a route '/about' mapped to an about() function that returns an about message.
4. Run the Flask application using app.run() when the script is executed directly.
5. Send GET requests to '/' and '/about' and verify the responses returned by the server.

Program

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Welcome to my Flask Web App!"

@app.route("/about")
def about():
    return "This is the About page."

if __name__ == "__main__":
    app.run(debug=True)
```

Output

```
Running: python p13.py
 * Running on http://127.0.0.1:5000 (Press CTRL+C to quit)

Tested using Flask's test client:
GET / -> 200  "Welcome to my Flask Web App!"
GET /about -> 200  "This is the About page."
```

Result / Conclusion

A simple Flask web application with multiple routes was successfully created and its endpoints were verified to return the correct responses.

Viva Questions

Q1. What does the @app.route() decorator do in Flask?

Ans: @app.route('/path') is a Python decorator that registers the decorated function as the handler (view function) for the specified URL path. When a client sends an HTTP request matching that path, Flask's URL dispatcher (Werkzeug's routing system) calls the corresponding view function and returns its response. By default, only GET requests are accepted; add methods=['GET','POST'] to handle multiple HTTP methods.

Q2. What is the difference between Flask and Django?

Ans: Flask is a micro-framework: minimal built-in features (routing, templating, development server), highly flexible, you add only what you need (ORM, auth, etc. are separate packages). Django is a full-stack framework: batteries-included with ORM, admin panel, authentication, forms, migrations out of the box. Flask is better for small apps, microservices, and REST APIs. Django is better for large, database-heavy web applications where convention over configuration is preferred.

Q3. What is WSGI and why is it important for Flask?

Ans: WSGI (Web Server Gateway Interface) is a Python standard (PEP 3333) that defines how web servers communicate with Python web applications. Flask is built on Werkzeug, a WSGI toolkit. A WSGI server (Gunicorn, uWSGI) calls the Flask app as a Python callable, passing the request environment and a response callback. This standard interface allows Flask apps to run on any WSGI-compatible server in production, decoupling the framework from the server.

Q4. Why should debug=True never be used in production?

Ans: debug=True enables: (1) automatic server restart on code changes; (2) the Werkzeug interactive debugger — an in-browser Python REPL that appears on unhandled exceptions. The interactive debugger allows arbitrary Python code execution in the server's process, which is a critical security vulnerability. Any visitor who triggers an error page could execute system commands. In production, use debug=False and a WSGI server like Gunicorn with proper error logging.

Q5. How does Flask's if __name__ == '__main__' guard work?

Ans: When Python runs a file directly (python app.py), __name__ is set to '__main__'. When the file is imported as a module (import app), __name__ is the module name ('app'). The guard ensures app.run() is only called when the script is executed directly, not when imported (e.g., by a WSGI server or test suite that imports the app object without wanting to start the server).

Practical 14: Database Integration with Backend Application

Aim

To study and implement database integration with a Python backend application using SQLite.

Theory

SQLite is a lightweight, serverless, self-contained, file-based (or in-memory) relational database engine that requires no separate database server process or configuration, making it ideal for embedded applications, prototypes, and testing; it is bundled with Python's standard library via the sqlite3 module, so no external installation is needed. It allows applications to create tables with defined schemas (column names and data types), insert, update, delete, and query records using standard SQL (Structured Query Language) statements. A Connection object, created via sqlite3.connect(), represents an open session with the database (which can be a file path or the special value ':memory:' for a temporary, non-persistent in-memory database, as used in this practical), and is responsible for committing or rolling back transactions via conn.commit(). A Cursor object, obtained from the connection, is used to execute SQL commands (cur.execute()) and iterate over or fetch the results of queries (fetchone(), fetchall()), acting as a control structure over the result set. Backend database integration like this forms the persistence layer of most real-world applications, storing data that must survive beyond a single program execution.

Algorithm

1. Import the sqlite3 module and create an in-memory database connection.
2. Create a cursor object from the connection.
3. Execute a CREATE TABLE statement to define the students table schema.
4. Execute INSERT statements to add student records and commit the transaction.
5. Execute a SELECT statement to retrieve all records and print them.
6. Close the database connection.

Program

```
import sqlite3

conn = sqlite3.connect(":memory:")
cur = conn.cursor()

cur.execute("""
    CREATE TABLE students (id INTEGER PRIMARY KEY, name TEXT, marks INTEGER)
""")

cur.execute("INSERT INTO students (name, marks) VALUES ('Amit', 85)")
cur.execute("INSERT INTO students (name, marks) VALUES ('Priya', 92)")
conn.commit()

cur.execute("SELECT * FROM students")
print("Database Records:")
for row in cur.fetchall():
    print(row)

conn.close()
```

Output

```
Database Records:
(1, 'Amit', 85)
(2, 'Priya', 92)
```

Result / Conclusion

Database integration with a Python backend was successfully implemented using SQLite, with table creation, insertion, and retrieval of records verified.

Viva Questions

Q1. What is the difference between SQLite and MySQL/PostgreSQL?

Ans: SQLite is serverless, file-based (or in-memory), zero-configuration, and single-process — ideal for embedded use, mobile apps, and prototyping. MySQL and PostgreSQL are client-server databases requiring a separate running server process, supporting multiple concurrent connections, user authentication, and large-scale production workloads with better performance, concurrency control (MVCC in PostgreSQL), and advanced features. SQLite is not suitable for high-concurrency web applications.

Q2. What is the purpose of the cursor in sqlite3?

Ans: A cursor (cur = conn.cursor()) is an interface to execute SQL commands and fetch results. cur.execute(sql) sends a query to the database engine. cur.fetchone() returns the next row as a tuple (or None). cur.fetchall() returns all remaining rows as a list of tuples. The cursor maintains the current position in the result set, allowing row-by-row iteration. A connection can have multiple cursors for parallel queries.

Q3. Why is conn.commit() necessary after INSERT/UPDATE/DELETE?

Ans: SQLite (and most relational databases) use transactions — a group of statements treated as an atomic unit. Changes made by INSERT/UPDATE/DELETE are held in a pending transaction in memory until conn.commit() makes them permanent (durable) in the database file. Without commit(), the changes are lost when the connection closes. conn.rollback() discards the pending changes. SELECT queries do not need commit() as they do not modify data.

Q4. What is the difference between ':memory:' and a file-path database?

Ans: sqlite3.connect(':memory:') creates a temporary database entirely in RAM — extremely fast, but all data is lost when the connection closes or the program exits. It is ideal for testing and temporary computation. sqlite3.connect('mydb.sqlite') creates or opens a persistent database file on disk — data survives program restarts. For production applications, always use a file-based database.

Q5. What is SQL injection and how can it be prevented in sqlite3?

Ans: SQL injection occurs when user-supplied input is concatenated directly into an SQL string, allowing an attacker to insert malicious SQL code. Example: f"SELECT * FROM users WHERE name='{name}'" — if name is "' OR '1'='1", it returns all rows. Prevention: always use parameterised queries with placeholders: cur.execute('SELECT * FROM users WHERE name=?', (name,)). The sqlite3 module safely escapes the value, preventing injection.

Practical 15: CRUD Operations in a Full-Stack Web Application

Aim

To study and implement Create, Read, Update, and Delete (CRUD) operations on a database using Python and SQLite.

Theory

CRUD refers to the four fundamental operations performed on persistent data in virtually any data-driven application: Create (insert new records into the database, corresponding to the SQL INSERT statement and typically an HTTP POST request in a web API), Read (retrieve one or more existing records, corresponding to SQL SELECT and an HTTP GET request), Update (modify the values of existing records, corresponding to SQL UPDATE and an HTTP PUT/PATCH request), and Delete (remove records, corresponding to SQL DELETE and an HTTP DELETE request). These four operations form the backbone of full-stack web applications: the frontend (UI) captures user actions, the backend (server-side application logic, e.g., Flask or Express) receives requests and translates them into the appropriate database operations, and the database (here, SQLite) persists the data using SQL statements executed through a database cursor, with every write operation (Create/Update/Delete) requiring an explicit commit() to make the change durable. Understanding CRUD is essential because nearly every real-world feature - user registration, a shopping cart, a content management system - is ultimately composed of these same four operations applied to different types of data.

Algorithm

1. Connect to an SQLite in-memory database and create a products table.
2. Create: insert new product records and commit the transaction, then display all records.
3. Read: query and display a specific product record by its id.
4. Update: modify the price of a specific product and commit, then display all records.
5. Delete: remove a specific product record and commit, then display the remaining records.
6. Close the database connection.

Program

```
import sqlite3

conn = sqlite3.connect(":memory:")
cur = conn.cursor()
cur.execute("CREATE TABLE products (id INTEGER PRIMARY KEY, name TEXT, price INTEGER)")

# Create
cur.execute("INSERT INTO products (name, price) VALUES ('Pen', 10)")
cur.execute("INSERT INTO products (name, price) VALUES ('Book', 50)")
conn.commit()
print("After CREATE:")
for row in cur.execute("SELECT * FROM products"):
    print(row)

# Read
print("\nREAD product id=1:")
print(cur.execute("SELECT * FROM products WHERE id=1").fetchone())

# Update
cur.execute("UPDATE products SET price=15 WHERE name='Pen'")
conn.commit()
print("\nAfter UPDATE:")
for row in cur.execute("SELECT * FROM products"):
    print(row)
# Delete
```

```
cur.execute("DELETE FROM products WHERE name='Book'")
conn.commit()
print("\nAfter DELETE:")
for row in cur.execute("SELECT * FROM products"):
    print(row)

conn.close()
```

Output

```
After CREATE:
(1, 'Pen', 10)
(2, 'Book', 50)

READ product id=1:
(1, 'Pen', 10)

After UPDATE:
(1, 'Pen', 15)
(2, 'Book', 50)

After DELETE:
(1, 'Pen', 15)
```

Result / Conclusion

CRUD operations (Create, Read, Update, Delete) were successfully implemented and verified on an SQLite database, forming the core of a full-stack web application's backend logic.

Viva Questions

Q1. What do CRUD operations map to in SQL and HTTP?

Ans: Create → SQL INSERT, HTTP POST. Read → SQL SELECT, HTTP GET. Update → SQL UPDATE, HTTP PUT (full replace) or PATCH (partial update). Delete → SQL DELETE, HTTP DELETE. This mapping is the foundation of RESTful API design: each resource (e.g., /products) exposes these four operations via the corresponding HTTP methods on its URL.

Q2. What is the difference between SQL UPDATE and DELETE?

Ans: UPDATE modifies the values of one or more columns in existing rows that match a WHERE condition — the rows remain in the table. DELETE removes entire rows that match a WHERE condition. Both require a WHERE clause in practice (omitting it affects ALL rows). Both are write operations requiring conn.commit() to persist. TRUNCATE (not in SQLite standard) removes all rows faster by dropping and re-creating the table internally.

Q3. What happens if you run UPDATE or DELETE without a WHERE clause?

Ans: Without a WHERE clause, UPDATE modifies every row in the table and DELETE removes every row. This is usually a catastrophic mistake in production. Always include a WHERE clause to target specific records (e.g., WHERE id=1). In production databases, use transactions so you can rollback if you accidentally run a destructive command without the WHERE clause.

Q4. What is a PRIMARY KEY and why is it important in CRUD?

Ans: A PRIMARY KEY is a column (or combination of columns) that uniquely identifies each row in a table. It must be NOT NULL and UNIQUE. In SQLite, INTEGER PRIMARY KEY auto-increments. In CRUD: the primary key is the target for Read (WHERE id=?), Update (SET ... WHERE id=?), and Delete (WHERE id=?) operations. Without a primary key, you cannot uniquely address individual records for update or deletion.

Q5. What is the difference between a full-stack and a backend-only application?

Ans: A backend-only application exposes data via APIs (JSON responses) but has no built-in user interface — clients (mobile apps, SPAs) consume the API separately. A full-stack application includes both the frontend (HTML/CSS/JS user interface, e.g., Jinja2 templates in Flask) and the backend (server logic, database) in one codebase. Full-stack covers the entire application from database to UI; backend-only is one tier of a multi-tier

architecture.